MENTOR REVIEW / 12 OCTOBER 2026

Campus Web House

Project review and preparation guide

Your project in one sentence

A full-stack campus guidance prototype: React provides the interface, Django processes requests, and SQLite stores colleges, discussions, answers and browser-session preferences.

Workspace: D:\College Project\Campus. The Python environment is D:\College Project\.venv, one directory above the project.

Architecture: Python Django backend + React/TypeScript frontend + SQLite. Django serves both the built frontend and its JSON endpoints.

How to prepare tonight

Read the directory explanations first. Practise starting the server and your own admin login. Rehearse one bookmark action and one question/answer submission. Finish by reading the mentor questions aloud.

Topic	Pages
Architecture and file walkthrough	2-5
Database, API and request flow	6-8
Importing data and running the project	9-10
Feature scope, security and testing	11-12
Live demo and mentor questions	13-15

Prepared from the actual local source on 11 October 2026. All eight backend tests passed during preparation. College figures, mentors and assessment content remain sample data, not live verified services.

REVIEW NOTES / 02

How the project works

Think of the system as three cooperating layers. The browser displays the interface, Django handles server operations, and SQLite remembers persistent information.

```
Browser / React     Django / portal         SQLite
Screens + actions -> URLs + views + ORM -> Tables + records
                  <- JSON response      <- Query result

GET /                      HTML page shell
/static/frontend/          Built JavaScript and CSS
GET /api/bootstrap/        Initial database + session state
POST /api/action/          A student action to persist
```

Django MVT in this project

Model: classes in portal/models.py define the data. **View:** functions in portal/views.py process requests and return HTML or JSON. **Template:** templates/index.html supplies the built page shell. React renders the screens within that page.

This is a Django backend with a single-page React interface. There are no separate Django page templates for each screen, and Django REST Framework is not used.

Roles of the technologies

Django: ORM, migrations, admin, sessions and CSRF handling. **React:** interactive screens, components and temporary interface state. **TypeScript:** compile-time type checks. **SQLite:** local single-file persistence. **Vite:** frontend bundling. **Tailwind CSS:** visual styling.

What happens when the home page opens

Django renders the HTML and sets a CSRF cookie. The browser loads compiled assets. React requests bootstrap data. Django reads models and browser-session state from SQLite and returns JSON. React uses that data to render the catalogue and community.

Node.js is required to install or rebuild frontend assets. Once assets exist, normal local startup uses Python and Django.

REVIEW NOTES / 03

Directory map

```
D:\College Project\
|-- .venv/                    Python environment
`-- Campus/                  Main application directory
    |-- manage.py
    |-- campus_project/
    |   |-- settings.py / urls.py
    |   `-- wsgi.py / asgi.py / __init__.py
    |-- portal/
    |   |-- models.py / views.py / urls.py / admin.py
    |   |-- tests.py / demo_data.json / __init__.py
    |   |-- migrations/0001_initial.py
    |   `-- management/commands/seed_demo.py
    |-- src/
    |   |-- main.tsx / App.tsx / api.ts / types.ts
    |   |-- index.css
    |   |-- screens/           Six screen components
    |   |-- components/        Header, BottomNav, Modals/
    |   `-- data/mockData.ts
    |-- scripts/               Build, export, cleanup tools
    |-- templates/index.html   Generated page shell
    |-- static/frontend/assets/  Generated JS and CSS
    |-- dist/                  Vite output
    |-- node_modules/          Installed Node dependencies
    |-- db.sqlite3             Actual local database
    |-- index.html             Source frontend shell
    |-- vite.config.ts / tsconfig.json
    |-- package.json / package-lock.json
    |-- requirements.txt / .env.example / .gitignore
    |-- README.md
    `-- output/pdf/            This preparation guide
```

The four folders to remember

campus_project: global Django configuration. **portal:** campus backend features. **src:** browser interface source. **templates + static:** compiled interface served by Django.

Source, generated files and stored data

Edit source files, then rebuild. Do not edit compiled JavaScript in static/ or dist/. node_modules, dist and Python caches are generated. Migrations are source history. db.sqlite3 contains real records and must not be treated as a disposable cache.

REVIEW NOTES / 04

Backend file responsibilities

File	Role
manage.py	Command entry point for server startup, migrations, tests, seeding and admin-account creation.
campus_project/settings.py	Installed apps, middleware, database, templates, static files, allowed hosts and environment configuration.
campus_project/urls.py	Top-level routes: / for home, /api/ for portal routes and /admin/ for administration.
wsgi.py and asgi.py	Integration entry points for server hosting; they do not implement college or discussion logic.
portal/models.py	Defines College, Discussion, Answer, VisitorState and MentorQuestion.
portal/views.py	Loads session-aware data, validates input, executes actions and returns HTML or JSON.
portal/urls.py	Connects bootstrap, action and thread-answer URLs to their view functions.
portal/admin.py	Registers models with admin; adds search/list settings for colleges and discussions.
portal/migrations/	Versioned instructions for creating or changing database structure.
portal/tests.py	Eight tests for page loading, persistence, submissions, validation, CSRF, voting and seed behavior.
portal/demo_data.json	Input sample data for importing; this is not the live database.
management/commands/seed_demo.py	Custom Django command that imports sample records through the ORM.

Django project versus Django app

The project is the overall configuration container. An app is a feature module inside it. campus_project configures this deployment; portal implements campus features. A project can contain several apps.

__init__.py identifies Python packages. Package markers also make the migrations and custom command directories discoverable.

REVIEW NOTES / 05

Frontend and build responsibilities

File / folder	Role
src/main.tsx	Starts React, mounts App into the root element and imports the global stylesheet.
src/App.tsx	Coordinates navigation, shared state and modal visibility. Calls Django to load data and persist actions.
src/api.ts	Shared fetch helper. GET for bootstrap, POST for actions; sends cookies and CSRF header; surfaces errors.
src/types.ts	College, discussion, answer, mentor and screen types, partly inferred from sample schemas.
src/index.css	Tailwind import, global styles, color tokens and fonts.
src/screens/	HomeScreen, DashboardScreen, CareersScreen, CollegesScreen, CommunityScreen and DiscussionDetailScreen.
src/components/	Header, BottomNav and reusable modals/drawers: assessment, ROI, compare, details, filters, profile and questions.
src/data/mockData.ts	Original colleges, discussions, answers, roadmaps, deadlines, resources and mentors.
index.html	Source page shell; compiled version is copied into templates/index.html.
vite.config.ts; tsconfig.json	React/Tailwind build plugins, asset URL base and API proxy; TypeScript configuration.
package.json; package-lock.json	Commands and dependency declarations; resolved dependency tree for reproducible installation.
scripts/build-django.mjs	Copies successful build assets into Django and replaces old hashed assets.
export-demo.mjs; clean.ps1	Exports sample data to JSON; cleans dist and Python caches in source folders.

Screen navigation uses React state; the individual screens do not have separate Django page URLs. College search, sorting and comparison operate on loaded data in the browser.

REVIEW NOTES / 06

Database models

Model	Important fields	Meaning
College	Slug ID, name, location, decimal annual_fee, JSON data	Catalogue entry. Annual fees use lakh units in the sample dataset.
Discussion	Slug ID, title, body, JSON data, created_at	A community thread and its display metadata.
Answer	Slug ID, discussion foreign key, JSON data, created_at	One answer belongs to one discussion.
VisitorState	Unique session_key, JSON data	Session bookmarks, vote flags, reminders, profile score and selected track.
MentorQuestion	session_key, mentor text, question, created_at	Stored question for admin review; no automated delivery.

```
Discussion 1 -------- many Answer
                  ForeignKey(on_delete=CASCADE)

Django session ---- VisitorState.session_key
                logical text link; not a ForeignKey
Django session ---- MentorQuestion.session_key
                logical text link; not a ForeignKey
```

Where the data lives

Campus/db.sqlite3 is the database. Django ORM converts Python model operations into database queries. Tables include portal_college, portal_discussion, portal_answer, portal_visitorstate and portal_mentorquestion. Django also creates authentication, session, admin-log and migration tables.

JSONField: purpose and tradeoff

JSON preserves flexible original UI data during the conversion. Core college fields and the answer relation are separately defined. Later, cutoffs, tags, votes and profiles could become dedicated relational models for stronger integrity and easier querying.

Inspected snapshot, 11 October 2026: 8 colleges, 3 discussions, 2 answers, 0 mentor questions and 3 visitor-state records. Counts may change with use.

REVIEW NOTES / 07

Routes and API actions

Route	Purpose
GET /	Render the page shell and issue a CSRF cookie.
GET /api/bootstrap/	Return all colleges/discussions and session-specific state.
POST /api/action/	Execute a JSON action and return refreshed application state.
GET /api/discussions/<id>/answers/	Return only that thread's answers and session vote/helpful flags.
/admin/	Administration interface requiring an authorized admin account.

Actions accepted by the POST endpoint

Action	Effect / payload
save_college; save_discussion	id: toggle session bookmark for an existing record.
vote_discussion	id: toggle vote flag and update shared upvote count.
create_discussion	title, body, tags: create thread with a UUID-based ID.
create_answer	id (discussion), body: create a linked answer.
vote_answer; helpful_answer	id (answer): toggle flags and change counters.
reminder	id: save deadline reminder flag, without scheduling delivery.
profile; read_notifications	Set demo profile score to 100; set unread count to zero.
assessment	track: save the selected career-track label.
mentor_question	question, mentor: save question for admin review.

Important HTTP responses

400: invalid JSON/content or unknown action. **404:** referenced record missing. **405:** wrong HTTP method. **403:** missing/invalid CSRF token. The frontend helper turns failed requests into error messages.

REVIEW NOTES / 08

Explain the Save College workflow

1. Student clicks Save

CollegesScreen invokes an App callback with the college slug ID.

2. React submits the action

App calls persist(), then api(). The fetch helper sends POST JSON, browser cookies and X-CSRFToken.

3. Django routes and validates

The project URL file delegates /api/ to portal routes. action() checks the request method and parses a JSON object.

4. Django finds the browser state

visitor() ensures a session exists and retrieves or creates VisitorState for its key. Preferences belong to the session, not a student login.

5. Django saves the flag

The college is checked with get_object_or_404. The bookmark value is toggled under VisitorState.data["colleges"] and saved through the ORM.

6. Django returns the updated list

state_payload() merges catalogue data with session preferences, and JsonResponse sends the result to React.

7. React renders and reload proves persistence

applyState() replaces shared frontend state. A later bootstrap request reads the same saved preference from SQLite.

```
{
  "action": "save_college",
  "id": "iit-delhi"
}
```

Demo proof: toggle a bookmark and refresh. A private browser session has separate preferences; sample initial flags may already be true, so compare the actual before/after state.

Question/answer flow: community submissions create shared Discussion and Answer records. Answer stores the discussion foreign key; bookmarks are session preferences rather than shared catalogue edits.

REVIEW NOTES / 09

Migrations and importing data

Structure and records are different

Migrations create or change database structure. They do not automatically import the college catalogue. **Seeding imports records** into existing tables.

```
# From Campus after activating the Python environment:
python manage.py migrate
python manage.py seed_demo
```

When you change models.py

```
python manage.py makemigrations portal
python manage.py migrate
```

makemigrations generates the schema-change file; migrate applies pending files. Keep migration files with source code. Neither command is needed for every normal restart.

How sample frontend data reaches the database

```
src/data/mockData.ts
       | npm run seed:export
       v
portal/demo_data.json
       | python manage.py seed_demo
       v
Django ORM -> SQLite records
```

The export script transforms TypeScript sample values to JSON. The seed command imports colleges, discussions and answers. Normal seeding uses get_or_create: running it repeatedly does not duplicate the same IDs or replace existing sample rows.

If you intentionally change the original sample dataset

```
npm run seed:export
python manage.py seed_demo --refresh
```

--refresh replaces matching bundled sample records with update_or_create, including their metadata. Use it deliberately. New questions and answers from the app already save through the API; they do not need a sample-data export.

Admin and backup

Create your own account with python manage.py createsuperuser, then visit /admin/. No default credentials are supplied. Before major database changes, stop the server and copy db.sqlite3. Database records are not routine cleanup files.

REVIEW NOTES / 10

Run the project: beginner steps

Open PowerShell. Paste each line separately and press Enter. The following path matches this computer; .venv is one directory above Campus.

```
cd "D:\College Project\Campus"
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
& "..\.venv\Scripts\Activate.ps1"
python manage.py runserver
```

After activation, the prompt starts with **(.venv)**. Open **http://127.0.0.1:8000/**. Keep the terminal open. Ctrl+C stops the server. The execution-policy setting applies only to this terminal process.

Command after activation	Purpose
python manage.py createsuperuser	Create your own admin username/password.
python manage.py migrate	Apply pending schema changes.
python manage.py seed_demo	Import missing sample records.
python manage.py check	Check Django configuration.
python manage.py test	Run tests from Campus.
npm run lint	TypeScript checking; not an ESLint run.
npm run build	Update compiled Django frontend assets.
npm run clean	Clean dist and source Python caches; retain DB/static assets.

Common errors

Activation script missing: check you are in Campus and that ..\.venv\Scripts\Activate.ps1 exists. The missing activation files were repaired in this session.

Port in use: another server may be running; open its URL or use python manage.py runserver 8001 and visit port 8001. **Django not found:** check the active (.venv) environment.

UI edits missing: run npm run build and refresh. **0 tests found:** run from Campus, or explicitly use python manage.py test portal. **Images/fonts missing:** some original assets require internet access.

REVIEW NOTES / 11

What works, and what is a prototype

Feature	Actual behavior
College catalogue	Database-backed records; browser search, sort, filter, details and compare. Compare selection is temporary React state.
Bookmarks and voting	College/thread bookmarks and vote/helpful flags persist per session. Counters are stored in shared record metadata.
Community	New questions and answers persist in SQLite. Each discussion loads its own answer list.
Student profile	Demo display data. Capstone action sets score to 100; no actual project upload or student registration.
Reminders / notifications	Reminder flags and unread count persist. No timed scheduler, email, push delivery or real notification inbox.
Mentor questions	Stored for admin review; no message delivery, automated assignment or response-time guarantee.
Career assessment	Wizard returns a fixed Full Stack Web Developer result. Track label persists, but App does not restore it into a dynamic roadmap. No live AI calls.
Roadmap controls	Sample content. Tasks/bookmarks use local React state; PDF-download and sandbox controls show demo toasts.
Reports / verification	Report submission is a demo interaction. Catalogue/placement verification labels are sample UI claims, not evidence of a live service.

ROI calculator formula in the code

Investment = (annual tuition + living costs) x course years, in lakhs. In-hand estimate = expected CTC x 0.76. Monthly savings = max(monthly in-hand - INR 35,000, INR 10,000). Payback = max(round(investment in rupees / monthly savings), 1).

Five-year multiple = annual in-hand x 5 x 1.15 / investment. These are illustrative fixed assumptions, not tax calculations, compounded salary growth or guaranteed outcomes. Future work should validate zero/negative inputs and no-positive-savings cases.

REVIEW NOTES / 12

Security and test evidence

Re-run during preparation on **11 October 2026**: Django configuration checks passed; **all 8 backend tests passed**. These verify the listed cases, not every interface or production scenario.

Test	Evidence
Page / bootstrap	HTML, CSRF cookie and database-backed catalogue count.
Session bookmarks	Persistence across requests and isolation from another browser session.
Question / answer	New thread starts empty; posted answer links to that thread.
Validation / methods	Reject invalid content, wrong methods, missing records and unknown actions.
CSRF enforcement	Tokenless POST fails; valid cookie/header succeeds.
Vote round trip	Two toggles restore original vote count and flag.
Progress / mentor	Profile/unread state changes; mentor question row is created.
Idempotent seed	Repeated seeding keeps the college count unchanged.

Security measures present

CSRF middleware and X-CSRFToken protect writes. Selected lengths and methods are validated. Admin uses Django authentication and password validators. Session cookies are HTTP-only and SameSite=Lax. Non-debug configuration requires a secret key and enables secure session/CSRF cookies.

Limits to explain honestly

Student authorization, rate limiting, moderation and production-scale testing are not implemented. JSON metadata and default sample vote flags are prototype choices. transaction.atomic groups changes, but select_for_update does not provide row-level locking on SQLite; do not claim concurrency-safe vote totals are proven.

Production would need HTTPS, an appropriate WSGI/ASGI server and static-file hosting. Set DJANGO_DEBUG, DJANGO_SECRET_KEY and DJANGO_ALLOWED_HOSTS in the process environment; .env.example is documentation and is not automatically loaded.

Commands to demonstrate

```
npm run lint
npm run build
python manage.py check
python manage.py test portal
```

REVIEW NOTES / 13

Presentation and live demo

Your 60-second opening

"Campus Web House is a campus and career guidance prototype. Students can explore and compare colleges, follow sample roadmaps and discuss questions. I converted the frontend-only application into a Django-backed project while preserving the React interface. Django models store colleges, discussions, answers and session preferences in SQLite. The frontend communicates through JSON endpoints. Django admin provides content management and review of stored mentor questions. The backend persistence and CSRF flows have tests. Student accounts, live feeds and personalized assessment are future work."

Suggested seven-minute review

Time	Show	Explain
0-1 min	Home + folder map	Purpose, three layers and project/app distinction.
1-2 min	Colleges	Search, details and compare; figures are sample data.
2-3 min	Bookmark + refresh	API action, session key and persistent saved flag.
3-4.5 min	Community	Create a review-demo question, open it, post an answer. Explain the foreign key.
4.5-5.5 min	Django admin	Show the records using your own admin account.
5.5-6.5 min	models.py + views.py	Show models, action dispatcher and state_payload().
6.5-7 min	Test results	Describe verified behavior and remaining limitations.

Checklist before the review

- Confirm activation and startup work. Use one server per chosen port.
- Verify your own admin login today. Password characters are hidden in terminal input.
- Use a clearly named demo question; submissions create real database records.
- Keep the editor, browser and this PDF ready.
- Rehearse the save-college request without reading every code line.
- If remote images fail, focus on database workflows and explain the dependency.

What to open in the editor

Start with the folder tree, then portal/models.py, both urls.py files, portal/views.py, src/api.ts and src/App.tsx. End with portal/tests.py. This follows the data flow and is easier to explain than opening files at random.

REVIEW NOTES / 14

Mentor questions: fundamentals

What problem does the project solve?

It combines college exploration, career-roadmap content and a student discussion space in one prototype.

Is the whole project Python?

The backend is Python Django; the interface is React and TypeScript. Django serves built assets and provides database actions.

What is a Django project versus an app?

campus_project holds global configuration. portal is the campus feature module. One project can contain multiple apps.

How does the frontend communicate with Django?

GET loads bootstrap data and answers. POST JSON sends actions. Cookies identify the browser session; X-CSRFToken accompanies writes.

Which database relation can you demonstrate?

A discussion has many answers. Answer.discussion is a foreign key with CASCADE deletion.

What is migrate versus seed_demo?

migrate applies schema changes. seed_demo imports bundled records into existing tables. Neither is a synonym for saving a new question.

How do bookmarks survive refresh?

VisitorState stores session flags keyed by record IDs. Bootstrap merges them with catalogue data and returns the saved values.

What does refreshing prove?

It checks that the UI can retrieve server-persisted state for that session rather than only display a temporary toast.

Why is Django REST Framework absent?

Plain Django views and JsonResponse currently serve a small endpoint set. A larger API could add structured API tooling later.

REVIEW NOTES / 15

Mentor questions: tradeoffs and revision

Why SQLite and JSON fields?

SQLite simplifies local setup. JSON preserves the original flexible UI data. More normalized models and a different database can be considered when integrity and concurrency requirements grow.

Does the project have student login?

No. Student identity is demo display data and preferences are session-based. Django admin authentication exists. Student accounts and ownership rules are future work.

Is the assessment AI-powered?

No. The wizard currently returns a fixed Full Stack Web Developer result. There is no live AI recommendation service.

Where does a mentor question go?

It becomes a MentorQuestion row for admin review. There is no automatic email or real mentor-response guarantee.

What were the main conversion challenges?

Preserving the UI while replacing temporary frontend state with database actions, linking answers to the correct thread, handling CSRF and serving Vite outputs through Django.

How do you verify the system?

TypeScript checks, frontend build, Django checks and eight backend tests. Separately demonstrate refresh persistence, community submissions and admin records.

What are the next improvements?

Student accounts and permissions, normalized votes/profiles, better input validation, moderation/reminders, reliable data sourcing and more integration/concurrency tests.

Remember these five statements

React shows. Django processes. SQLite remembers. Migrate creates structure. Seed_demo imports sample records. Be ready to identify models.py, views.py, both urls.py files, App.tsx, api.ts and db.sqlite3.

Source basis: local README.md; campus_project settings/routes; portal models, views, routes, admin, tests and seed command; src/App.tsx, api.ts, types.ts, CareersScreen.tsx, RoiCalculatorModal.tsx and CareerAssessmentModal.tsx; package.json; vite.config.ts; build-django.mjs. Source, database counts and test results inspected 11 October 2026.